

ENVIRONMENT DIAGRAM Meta

COMPUTER SCIENCE MENTORS 61A

March 6, 2026

Recommended Timeline

- Ice Breaker/Get to know each other [10 minutes]
- What Would Python Display + Minilecture/explanations [15 minutes]
- Operators and Operands [5 minutes]
 - Note: don't need to do all parts if students get the idea
- Code Writing [20 minutes]
 - Again, no need to do all questions
 - 2 and 3 are medium difficulty & the last one is a bit more challenging, pick based on how your students are feeling
- Tips and Tricks for succeeding in CS61A [leftover time]

1 Environment Diagram

1. Draw the environment diagram that results from running the code.

```
six = [6]
seven = [7]

def six_seven(sixty, seventy):
    def six_or_seven(s):
        if s.pop() == 6:
            return 67
        else:
            return 7
    seven = six_or_seven(sixty)
    sixty.append(seven.extend(six))
    sixty.append([seven])
    return sixty

ss = six_seven(six, seven)
print(ss)
```

<https://tinyurl.com/4a7dyxww>

2. You are at a buffet! Yummy.

You see that the buffet line has many different dishes: p (protein), v (vegetable), r (rice), and f (fruit). You note down the items in line as ****options****.

Now, you are very picky about the order in which food reaches your plate and want to get food ****in a fixed order****, but you are also very indecisive and want to know if different orders of getting food is possible.

Complete the ****buffet_possible**** function, that takes in the buffet options (options) and your desired choice (choice), and returns whether it is possible to get your choice from the line.

```
def buffet_possible(options, choice):
    """
    >>> buffet_possible("pvprprpff", "p")
    True
    >>> buffet_possible("pvprprrvpff", "pvpvp")
    True
    >>> buffet_possible("pvprprrvpff", "fff")
    False
    >>> buffet_possible("pvprprrvpff", "")
    True
    """

    if _____:
        return _____
    elif _____:
        return _____
    else:
        if _____:
            return _____
        else:
            return _____

def buffet_possible(options, choice):
    if choice == "":
        return True
    elif options == "":
        return False
    else:
        if options[0] == choice[0]:
            return buffet_possible(options[1:], choice[1:])
        else:
            return buffet_possible(options[1:], choice)
```

3. Finally, you have decided what to eat.

Now, given **options** and **choice**, you want to know how you should get food from the line.

Complete the **buffet** function, that takes in the buffet options (options) and what you want (choice), and returns the list containing lists of possible positions to take the dishes.

```
def buffet(options , choice):
    """
    >>> buffet("pvprprpff", "p")
    [[0], [2], [4], [7]]
    >>> buffet("pvprprvpff", "rr")
    [[3, 5], [3, 6], [5, 6]]
    >>> buffet("pvprprvpff", "pvprprvpff")
    [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]
    >>> buffet("rrfff", "rrrr") # Not possible
    []
    >>> buffet("rrfff", "") # Not selecting anything
    [[]]
    """

    def helper(options , choice , current_index):
        if _____:
            return [[]]
        if _____:
            return []
        with_current = _____
        without_current = _____
        if _____:
            return _____
        else:
            return _____

    return _____

def buffet(options , choice):
    def helper(options , choice , current_index):
        if choice == "":
            return [[]]
        if options == "":
            return []
        with_current = [[current_index] + x for x in helper(options[1:],
            choice[1:], current_index+1)]
        without_current = helper(options[1:], choice , current_index+1)
        if options[0] == choice[0]:
            return with_current + without_current
        else:
            return without_current

    return helper(options , choice , 0)
```